Algorithms & Complexities

`Algorithms and Complexities` play a fundamental role in this project, as algorithms are used throughout, and the complexities of these algorithms have a direct impact on the performance of the application.

Algorithms

- **BM25**: This algorithm is used to score items based on a search query. It gives a score to each item based on an advanced algorithm designed to give the most relevant results a higher score. The scores are then used to filter out items that are not relevant to the search query. Citations are in the files.
    - Used in the QueryFilter and MyBM25 classes.
    - Why it was chosen
        - Allowed for custom weighting of different fields.
        - Deals with overuse of terms with saturation.
        - Deals with long vs short content.
        - Widely used and lots of documentation on the algorithm and implementation in java.
    - Implementation Details
        - An analyzer was built that tokenized the query from 2 to 5 characters. This allowed for typos to not be penalized too hard, while still giving better scores to items with longer matches. The analyzer also removes stop words (common English words such as 'the', 'a', etc.).
        - Directory is all in RAM, as data is loaded in from objects in memory, and the access is faster.
        - When filtering, the highest score is found, and items with scores less than 15% of the highest score are filtered out. This allows for precise queries to have fewer results, while broad queries can have many results.
- **Sieve of Eratosthenes**: This algorithm is used to generate a prime number to use for the universal hashing in `ItemHashMap`. The algorithm is used in the SieveOfEratosthenes class. Original code is from the modules for the course.
    - Why it was chosen
        - I needed a large prime number for the universal hashing.
        - The code was simple to implement and alter to work for my needs.
    - Implementation Details
        - Uses a `BitSet` rather than a boolean array to save space, as the size of the sieve is quite large (100 million).
    - Complexity:
        - Time Complexity: $O(n \log \log n)$
        - Space Complexity: $O(n)$
- **Dijkstra's Algorithm**: This algorithm is used to find the shortest path between two items in the `SimilarItemsGraph`. It is used in the SimilarItemsGraph class. The implementation is based on the standard Dijkstra's algorithm, with some modifications to work with the `SimilarItemsGraph`. The use is to find the top similar items to another given item. Citations are in the files.
    - Why it was chosen

        - The goal I am looking for is the most similar items to my item, and this can be found easily with Dijkstra's algorithm.
    - Implementation Details
        - With 961 items, there are `961 choose 2` or 461280 possible edges. A script was run in Edge Analysis to find the 95th percentile of edge weights, and only edges with weights above this threshold were added to the graph. Filling the graph still takes a long time, as each edge requires a similarity score to be calculated before the map can reject the edge.
    - Complexity:
        - Time Complexity: `O((V + E) log V)` where `V` is the number of items and `E` is the number of edges in the graph.
        - Space Complexity: `O(V + E)` for the graph representation. `O(V)` for the priority queue and the distance map used in the algorithm.
- **Universal Hashing**: This is used in the `IdHashKey` class to generate hash codes for items based on their IDs. Before the hash code is generated, the ID is first collapsed into a number by a `Rabin-Karp` rolling hash. Original code for the hashing is from the modules for the course, but I altered it to work for my needs.
    - Why it was chosen
        - Universal hashing shows better performance in terms of collisions. While it is possible for a hash function here to have more collisions than the standard `String.hashCode()`, in many cases it gives fewer collisions.
    - Implementation Details
        - A large prime number is generated using the Sieve of Eratosthenes algorithm, and this prime number is used in the hash function. The hash code is generated by first collapsing the ID into a number using a Rabin-Karp rolling hash, and then applying the universal hashing formula to generate the final hash code.
    - Complexity:
        - Time Complexity: `O(n)` where `n` is the length of the ID string, as each character in the ID needs to be processed before universal hashing can be applied.
        - Space Complexity: `O(1)`. It saves a few numerical values temporarily during the algorithm.
- **Rabin-Karp Rolling Hash**: This is used in the `IdHashKey` class to collapse the ID string into a number before applying universal hashing. Citation is in the file.
    - Why it was chosen
        - A simple method was needed to convert the ID string into a number in order for universal hashing to be applied. The Rabin-Karp rolling hash is a simple and efficient way to do this.
        - It uses a single pass through the string.
    - Implementation Details
        - The allowed characters are all ASCII character from `!` to `~`. `BigInteger` is used, so overflow should be avoided even with larger numbers.
- **Red-Black Tree Search**: This is used in the `PriceTree` class to find item indexes based on price. The `PriceTree` class is a red-black tree that stores items based on their price, and allows for efficient searching of items within a certain price range.
    - Why it was chosen
        - The `TreeMap` class in Java already implemented the red-black tree and provides a `subMap` method that allows for efficient searching of items within a certain price range. This made it a convenient choice for implementing the price filter.
    - Implementation Details

        - Nodes in the tree store an item's price as the key, and an item index as the value. This allows for an array of item indexes to be returned. The algorithm here is just doing many traversals of the tree, with the successor algorithm being used.
    - Complexity:
        - Time Complexity: `O(log n)` where `n` is the number of unique prices in the tree.
        - Space Complexity: `O(n)` for storing the items in the tree. The search itself uses `O(k)` space to store the results.